PROPOSAL TUGAS AKHIR - EE234799

PERBANDINGAN KINERJA ALGORITMA FXLMS DAN FXNLMS PADA SISTEM *ACTIVE NOISE CANCELLATION* (ANC) *HEADPHONE* BERBASIS FPGA

KEINDRA DANTE HIDAYAT

NRP 5022231236

Dosen Pembimbing

Astria Nur Irfansyah, S.T., M.Eng., Ph.D.

NIP 198103252010121002

Program Studi Sarjana 1 (S1) Teknik Elektro

Departemen Teknik Elektro

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026

PROPOSAL TUGAS AKHIR - EE234799

PERBANDINGAN KINERJA ALGORITMA FXLMS DAN FXNLMS PADA SISTEM *ACTIVE NOISE CANCELLATION* (ANC) *HEADPHONE* BERBASIS FPGA

KEINDRA DANTE HIDAYAT

NRP 5022231236

Dosen Pembimbing

Astria Nur Irfansyah, S.T., M.Eng., Ph.D.

NIP 198103252010121002

Program Studi Sarjana 1 (S1) Teknik Elektro

Departemen Teknik Elektro

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026

[Halaman ini sengaja dikosongkan]

FINAL PROJECT PROPOSAL - EE234799

PERFORMANCE COMPARISON OF FXLMS AND FXNLMS ALGORITHMS IN AN FPGA-BASED HEADPHONE ACTIVE NOISE CANCELLATION (ANC) SYSTEM

KEINDRA DANTE HIDAYAT

NRP 5022231236

Advisor

Astria Nur Irfansyah, S.T., M.Eng., Ph.D.

NIP 198103252010121002

Undergraduate Study Program of Electrical Engineering

Department of Electrical Engineering

Faculty of Intelligent Electrical and Informatics Technology

Sepuluh Nopember Institute of Technology

Surabaya

2026

[Halaman ini sengaja dikosongkan]

ABSTRAK

Nama Mahasiswa : KEINDRA DANTE HIDAYAT
Judul Tugas Akhir : PERBANDINGAN KINERJA ALGORITMA FXLMS DAN FXNLMS PADA SISTEM *ACTIVE NOISE CANCELLATION* (ANC) *HEADPHONE* BERBASIS FPGA
Pembimbing : 1. Astria Nur Irfansyah, S.T., M.Eng., Ph.D.

Active Noise Cancellation (ANC) digunakan untuk meredam kebisingan frekuensi rendah yang tidak dapat diatasi secara efektif oleh peredam pasif. Algoritma *Filtered-X Least Mean Square* (FXLMS) menjadi standar pada sistem ANC, tetapi *step-size* yang tetap membuat kinerjanya bergantung pada *tuning* manual dan rentan terhadap variasi daya sinyal kebisingan *broadband*. *Filtered-X Normalized Least Mean Square* (FXNLMS) mengatasi kelemahan tersebut dengan menormalisasi *step-size* terhadap daya sinyal referensi terfilter, tetapi menambah estimasi daya dan operasi pembagian pada setiap iterasi. Penelitian terdahulu umumnya hanya menyoroti kinerja peredaman atau efisiensi perangkat keras secara terpisah, sehingga belum diketahui apakah keunggulan FXNLMS sepadan dengan biaya komputasi tambahannya. Penelitian ini membandingkan FXLMS dan FXNLMS pada sistem ANC *headphone feedforward single-channel* (SISO) melalui tiga tahap, yaitu simulasi kinerja peredaman menggunakan *white noise*, *pink noise*, dan rekaman bising lingkungan nyata; analisis pengaruh normalisasi *step-size* terhadap kecepatan konvergensi dan *steady-state error*; serta implementasi *fixed-point* pada FPGA Artix-7 XC7A35T. Kinerja dievaluasi melalui peredaman kebisingan (SNR), kecepatan konvergensi, dan *steady-state error*, sedangkan efisiensi komputasi dievaluasi melalui kompleksitas operasi per-iterasi serta utilisasi LUT, *flip-flop*, memori, dan DSP *slice*. Hasil penelitian diharapkan memberikan dasar kuantitatif dalam memilih algoritma ANC untuk perangkat tersemat berkapasitas terbatas.

Kata Kunci: *Active Noise Cancellation*, FXLMS, FXNLMS, FPGA, Efisiensi Komputasi

[Halaman ini sengaja dikosongkan]

ABSTRACT

Name : KEINDRA DANTE HIDAYAT
Title : *PERFORMANCE COMPARISON OF FXLMS AND FXNLMS ALGORITHMS IN AN FPGA-BASED HEADPHONE ACTIVE NOISE CANCELLATION (ANC) SYSTEM*
Advisors : 1. Astria Nur Irfansyah, S.T., M.Eng., Ph.D.

Active Noise Cancellation (ANC) is used to attenuate low-frequency noise that passive attenuation cannot suppress. The Filtered-X Least Mean Square (FXLMS) algorithm is the standard in ANC systems, but its fixed step-size makes its performance dependent on manual tuning and sensitive to power variations in broadband noise. The Filtered-X Normalized Least Mean Square (FXNLMS) algorithm addresses this weakness by normalizing the step-size to the power of the filtered reference signal, but adds a power estimate and a division per iteration. Previous studies generally examined either noise reduction performance or hardware efficiency separately, so it remains unclear whether FXNLMS justifies its additional computational cost. This research compares FXLMS and FXNLMS in a feedforward single-channel (SISO) headphone ANC system in three stages: noise reduction simulation using white noise, pink noise, and real-world ambient noise recordings; analysis of how step-size normalization affects convergence speed and steady-state error; and fixed-point implementation on an Artix-7 XC7A35T FPGA. Performance is evaluated through noise reduction (SNR), convergence speed, and steady-state error, while computational efficiency is evaluated through per-iteration operation complexity and the utilization of LUTs, flip-flops, memory, and DSP slices. The results are expected to give a quantitative basis for selecting ANC algorithms for resource-constrained embedded devices.

Keywords: *Active Noise Cancellation*, *FXLMS*, *FXNLMS*, *FPGA*, *Computational Efficiency*

[Halaman ini sengaja dikosongkan]

DAFTAR ISI

ABSTRAK i

ABSTRACT iii

DAFTAR ISI v

DAFTAR GAMBAR vii

DAFTAR TABEL ix

1 PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Rumusan Masalah 2

1.3 Batasan Masalah 2

1.4 Tujuan 2

1.5 Manfaat 3

1.5.1 Bagi Peneliti 3

1.5.2 Bagi Masyarakat Umum 3

2 TINJAUAN PUSTAKA 5

2.1 Hasil Penelitian Terdahulu 5

2.2 Dasar Teori 7

2.2.1 *Active Noise Cancellation* (ANC) 7

2.2.2 Filter Adaptif dan Algoritma LMS 9

2.2.3 Algoritma FXLMS 10

2.2.4 Algoritma FXNLMS 11

2.2.5 *Field-Programmable Gate Array* (FPGA) 13

3 METODOLOGI 15

3.1 Metode Penelitian 15

3.2 Alat dan Bahan Penelitian 15

3.3 Alur Pelaksanaan Penelitian 17

3.4 Perancangan Sistem ANC *Headphone* Berbasis FPGA 18

3.4.1 Arsitektur Sistem 18
3.4.2 Model Simulasi Sistem ANC 20
3.4.3 Estimasi Jalur Sekunder 20
3.4.4 Rancangan Algoritma FXLMS dan FXNLMS 21
3.4.5 Implementasi pada FPGA 23
3.5 Metode Pengujian dan Analisis Data 24
3.5.1 Pengujian Kinerja Peredaman 24
3.5.2 Pengujian Kecepatan Konvergensi dan *Steady-State Error* 24
3.5.3 Verifikasi Rancangan HDL dan Prototipe 25
3.5.4 Evaluasi Efisiensi Komputasi 25
3.5.5 Analisis Data Akhir 25

DAFTAR PUSTAKA **27**

DAFTAR GAMBAR

3.1 Diagram alir pelaksanaan penelitian 17
3.2 Diagram blok sistem ANC *headphone* berbasis FPGA 19
3.3 Skema estimasi jalur sekunder secara *offline* 21
3.4 Alur pemrosesan setiap sampel pada FXLMS dan FXNLMS 23

[Halaman ini sengaja dikosongkan]

DAFTAR TABEL

2.1 Hasil Penelitian Terdahulu 5
2.2 Perbandingan FXLMS dan FXNLMS 13

3.1 Alat dan bahan penelitian 15
3.2 Repositori sinyal bising lingkungan nyata 16
3.3 Jadwal pelaksanaan tugas akhir 18
3.4 Rancangan awal algoritma dan dugaan awal 22

[Halaman ini sengaja dikosongkan]

BAB I
PENDAHULUAN

1.1 Latar Belakang

Polusi suara mendegradasi kenyamanan akustik pada berbagai lingkungan, mulai dari kabin kendaraan hingga perangkat audio portabel. *Passive Noise Cancellation* (PNC) efektif meredam kebisingan frekuensi tinggi, namun performanya menurun drastis pada frekuensi rendah [1, 2]. *Active Noise Cancellation* (ANC) diterapkan untuk mengatasi keterbatasan tersebut melalui prinsip interferensi destruktif secara *real-time*.

Algoritma *Filtered-X Least Mean Square* (FXLMS) menjadi standar dalam ANC karena mampu mengompensasi jalur sekunder dan menjaga stabilitas konvergensi [2, 3]. Idealnya, algoritma kontrol adaptif pada sistem ANC memberikan konvergensi cepat dan stabil terhadap kebisingan *broadband* yang bersifat non-stasioner tanpa membebani sumber daya komputasi perangkat secara berlebihan, terutama pada perangkat tersemat berkapasitas terbatas seperti *headphone*. Pada praktiknya, FXLMS standar menggunakan *step-size* (μ) tetap sehingga kinerjanya bergantung pada *tuning* manual dan rentan terhadap variasi daya sinyal pada kebisingan *broadband* tersebut [4]. *Filtered-X Normalized Least Mean Square* (FXNLMS) dikembangkan untuk mengatasi kelemahan ini dengan menormalisasi *step-size* secara adaptif berdasarkan daya sinyal referensi terfilter. Pada aplikasi otomotif, pendekatan ini dilaporkan meningkatkan peredaman kebisingan hingga rata-rata 5,07 dB dengan performa terbaik di bawah 1 kHz, sekaligus mempercepat konvergensi dibandingkan FXLMS standar [1, 4].

Peningkatan performa tersebut disertai konsekuensi pada sisi komputasi. Normalisasi *step-size* pada FXNLMS menambah beban berupa estimasi daya sinyal dan operasi pembagian pada setiap iterasi, yang berdampak pada kompleksitas implementasi di perangkat tersemat berkapasitas terbatas [3, 4]. Sejauh ini, penelitian yang membandingkan FXLMS dan FXNLMS umumnya hanya menyoroti satu sisi dari persoalan tersebut: performa peredaman melalui simulasi perangkat lunak [1, 2, 3, 4], atau efisiensi sumber daya perangkat keras pada algoritma adaptif lain secara terpisah, seperti reduksi penggunaan sumber daya logika FPGA sebesar 12,5% hingga 53,3% pada implementasi *Signed-Error* LMS [5]. Belum ada kajian yang menempatkan kedua sisi ini secara berdampingan untuk kebisingan *broadband* pada sistem ANC *headphone*, sehingga belum diketahui secara empiris apakah keunggulan konvergensi FXNLMS sepadan dengan biaya komputasi tambahan yang ditimbulkannya.

Kesenjangan inilah yang mendasari tiga pertanyaan yang diajukan dalam penelitian ini. Pertama, sejauh mana FXNLMS unggul dibandingkan FXLMS standar dalam meredam kebisingan *broadband* pada sistem ANC *headphone*. Kedua, apakah normalisasi *step-size* benar-benar mempercepat konvergensi dan menurunkan *steady-state error* saat menghadapi fluktuasi sinyal *broadband*, mengingat *step-size* tetap pada FXLMS menghadirkan dilema klasik antara kecepatan dan kestabilan. Ketiga, seberapa besar tambahan beban komputasi yang harus ditanggung untuk memperoleh keunggulan tersebut, dilihat dari kompleksitas operasi per-iterasi maupun konsumsi sumber daya logika pada FPGA.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah dalam tugas akhir ini adalah sebagai berikut:

1. Bagaimana perbandingan kinerja algoritma FXLMS dan FXNLMS dalam meredam sinyal kebisingan tipe *broadband* pada sistem ANC *headphone*?
2. Bagaimana pengaruh mekanisme normalisasi *step-size* pada algoritma FXNLMS terhadap kecepatan konvergensi dan *steady-state error* saat menangani fluktuasi sinyal *broadband*, jika dibandingkan dengan algoritma FXLMS standar yang menggunakan *step-size* tetap?
3. Bagaimana tingkat efisiensi komputasi algoritma FXNLMS dibandingkan FXLMS standar, ditinjau dari kompleksitas operasi per-iterasi (jumlah operasi perkalian, penjumlahan, dan pembagian) serta utilisasi sumber daya logika blok DSP, MAC, dan LUT pada arsitektur FPGA, mengingat FXNLMS memerlukan operasi normalisasi tambahan?

1.3 Batasan Masalah

Batasan masalah dalam tugas akhir ini adalah sebagai berikut:

1. Topologi Sistem: Perancangan arsitektur *Active Noise Cancellation* (ANC) dibatasi pada sistem *Feedforward Single-Channel* (SISO - *Single-Input Single-Output*), yang merepresentasikan model pemrosesan akustik untuk satu sisi *earpiece* pada *headphone*.
2. Cakupan Algoritma: Algoritma filter adaptif yang diimplementasikan dan dikomparasikan secara eksklusif hanya algoritma *Filtered-X Least Mean Square* (FXLMS) standar dengan *step-size* tetap dan *Filtered-X Normalized Least Mean Square* (FXNLMS) dengan *step-size* ternormalisasi.
3. Karakteristik Sinyal Bising: Sinyal referensi atau gangguan yang dievaluasi difokuskan pada kebisingan spektrum lebar (*broadband noise*) pada pita frekuensi rendah (rentang frekuensi di bawah 100 Hz–1000 Hz). Sinyal uji direpresentasikan menggunakan *white noise*, *pink noise*, atau rekaman bising lingkungan nyata (*real-world ambient noise*).
4. Metrik Evaluasi Kinerja dan Efisiensi: Evaluasi kinerja dibatasi pada metrik peredaman kebisingan (*noise reduction*) yang diukur dalam SNR (*Signal-to-Noise Ratio*), kecepatan konvergensi, dan *steady-state error* hasil simulasi. Evaluasi efisiensi komputasi dibatasi pada kompleksitas operasi aritmetika per-iterasi serta metrik sintesis logika pada platform *Field-Programmable Gate Array* (FPGA), yaitu utilisasi sumber daya (LUT, *Flip-Flop*, memori, dan DSP *Slices*).

1.4 Tujuan

Berdasarkan rumusan masalah, tujuan dari penelitian ini adalah sebagai berikut:

1. Menganalisis secara komparatif kemampuan algoritma FXLMS dan FXNLMS dalam mereduksi sinyal kebisingan spektrum lebar pada sistem ANC *headphone*.

2. Mengukur dan mengidentifikasi pengaruh mekanisme normalisasi *step-size* pada algoritma FXNLMS terhadap performa metrik kontrol, khususnya pada kecepatan konvergensi (*convergence rate*) dan nilai galat kondisi tunak (*steady-state error*), jika dibandingkan dengan performa FXLMS standar.

3. Menghitung dan membandingkan tingkat efisiensi komputasi antara algoritma FXLMS dan FXNLMS, mencakup kompleksitas operasi per-iterasi serta konsumsi sumber daya perangkat keras digital LUT, *Flip-Flops*, dan DSP *Slices* yang dibutuhkan untuk mengimplementasikan koefisien filter *broadband* pada platform FPGA.

1.5 Manfaat

Manfaat yang diharapkan dari penelitian ini adalah sebagai berikut.

1.5.1 Bagi Peneliti

1. Meningkatkan pemahaman dan keterampilan dalam perancangan sistem *Active Noise Cancellation* (ANC), khususnya algoritma filter adaptif FXLMS dan FXNLMS, aritmetika *fixed-point*, serta implementasi dan sintesis HDL pada FPGA.

2. Menerapkan pengetahuan yang diperoleh selama pendidikan sarjana pada bidang pengolahan sinyal digital dan rekayasa elektronika.

1.5.2 Bagi Masyarakat Umum

1. Memberikan kontribusi pengetahuan mengenai perbandingan kinerja dan efisiensi komputasi algoritma FXLMS dan FXNLMS pada sistem ANC *headphone* berbasis FPGA, sebagai dasar kuantitatif dalam memilih algoritma ANC untuk perangkat tersemat berkapasitas terbatas.

[Halaman ini sengaja dikosongkan]

BAB II
TINJAUAN PUSTAKA

2.1 Hasil Penelitian Terdahulu

Penelitian mengenai sistem ANC berbasis algoritma keluarga LMS telah dilakukan dengan objek, metode, dan platform implementasi yang beragam. Untuk memetakan posisi penelitian ini, lima penelitian terdahulu yang paling relevan dirangkum pada Tabel 2.1 berdasarkan objek atau data yang digunakan, metode, parameter dan hasil utama, serta keterkaitannya dengan penelitian ini.

Tabel 2.1: Hasil Penelitian Terdahulu

No.	Peneliti	Objek/Data	Metode	Parameter dan Hasil Utama	Keterikatan dengan Penelitian
1.	Xie, Zhou & Xie (2022)	Derau acak (*random source*, f_s 60 kHz) pada model *feedforward* untuk *earphone* ANC	Simulasi FxLMS di MATLAB/Simulink, ditambah *low-pass filter* (LPF) FIR/Chebyshev orde 16 pada sinyal residu	Orde filter 5-30 dan *step size* 0,0001-1. Pada orde 8 dan μ = 0,5, STD derau 0,04797 turun menjadi 0,000778 (0,0003826 dengan LPF). Orde terlalu rendah tidak konvergen, sedangkan orde dan μ besar membuat sistem divergen	Menjadi acuan pengaruh orde filter dan *step size* terhadap kecepatan konvergensi dan stabilitas FxLMS pada sistem *headphone*. Masih berupa simulasi, implementasi *hardware* disebut sebagai penelitian lanjutan
2.	Han & Chen (2023)	Nada tunggal 200 Hz dan 300 Hz (±55 dB), serta perpindahan 200 Hz ke 300 Hz	*Hybrid fixed-filter* + FxLMS: FFT memilih koefisien pra-latih sebagai nilai awal FxLMS. Diimplementasikan pada mikrokontroler STM32H743	Orde 128, f_s 20 kHz, μ = 0,001. Reduksi ±15 dB (200 Hz) tidak berubah, tetapi waktu konvergensi 300 Hz turun dari 10 s menjadi 5 s. Pada simulasi (μ = 0,005), konvergensi dari ±50.000 sampel menjadi ±20.000 sampel	Menunjukkan kelemahan FxLMS konvensional (konvergensi lambat, beban komputasi) pada sistem digital nyata. Orde 128 dan f_s 20 kHz dapat menjadi pembanding parameter. Tidak membandingkan dengan NLMS dan tidak melaporkan penggunaan sumber daya

Tabel 2.1: Hasil Penelitian Terdahulu (lanjutan)

No.	Peneliti	Objek/Data	Metode	Parameter dan Hasil Utama	Keterikatan dengan Penelitian
3.	Voltolini et al. (2023*)	Derau struktural pembangkit kogenerasi; 3 akselerometer sebagai referensi, 3 *shaker*, 1 mikrofon *error* pada jarak 40 m	FxNLMS *offline* di MATLAB (SISO hingga SIMO multi-referensi), lalu uji *real-time hardware-in-the-loop* pada DSP ADAU1466Z	Rentang 50-250 Hz, orde W = 256, *downsampling* 16× (48 kHz ke 3 kHz), μ = 0,1. Reduksi 3,6-4,0 dB(A) (terbaik J = 2, K = 2: 4,0 dB(A)). Hasil *real-time* identik dengan *offline*, konvergensi ±7 s	Salah satu dari sedikit *paper* yang menjalankan FxNLMS pada perangkat keras khusus secara *real-time*. Teknik *downsampling* untuk menekan beban komputasi relevan bagi FPGA. Bukan aplikasi *headphone*, tanpa pembanding FxLMS dan tanpa laporan sumber daya DSP
4.	Chen, Yang, Lai & Wei (2025)	Derau impulsif frekuensi rendah (tembakan artileri) untuk perlindungan pendengaran	FxLMS arsitektur *feedback* (AANC) + *audio-balance control circuit* (ABCC); simulasi MATLAB R2023b dan implementasi rangkaian elektronik	Simulasi orde 30-60, μ antara 10^{-3} dan 10^{-2}. Konfigurasi terbaik pada *hardware* adalah orde 30 dan $\mu = 10^{-3}$. Reduksi maksimum 21,8 dB, dan 10-20 dB di bawah 2 kHz	Menegaskan bahwa orde dan *step size* harus di-*tuning* ulang saat berpindah dari simulasi ke *hardware*. Berbasis rangkaian elektronik (op-amp TL074/OPA810), bukan FPGA. Tidak ada FxNLMS dan tidak ada analisis sumber daya digital
5.	Cesur et al. (2025*)	Derau jalan di kabin kendaraan listrik (rekaman lintasan uji, 2 mikrofon di posisi kepala pengemudi)	FxNLMS dengan *variable step-size*, pemodelan jalur sekunder *online* (PRBS + NLMS), *high-pass filter* dan *soft limiter*	f_s 44,1 kHz, *buffer* 256 sampel (±5,8 ms). Reduksi rata-rata 5,07 dB (8,28 dB di 250 Hz, 7,31 dB di 750 Hz), turun menjadi 2,4-3,6 dB di atas 1 kHz. Dibanding FxLMS standar: konvergensi lebih cepat dan reduksi lebih besar (disebut kualitatif, tanpa angka)	Paling dekat dengan topik penelitian ini karena menyatakan FxNLMS lebih unggul dari FxLMS dan cocok untuk sistem *embedded* dengan daya komputasi terbatas. Perbandingannya tidak kuantitatif, bukan di FPGA, dan tidak ada laporan sumber daya

Dua penelitian pertama mengkaji FXLMS pada tingkat simulasi dan mikrokontroler. Xie et al. [3] mensimulasikan FXLMS pada model *feedforward* untuk *earphone* ANC di MATLAB/Simulink dan menunjukkan bahwa orde filter dan *step-size* menentukan kestabilan algoritma. Pada orde 8 dan $\mu = 0{,}5$, standar deviasi derau turun dari 0,04797 menjadi 0,000778, sedangkan orde yang terlalu rendah gagal konvergen dan kombinasi orde serta μ yang besar membuat sistem divergen. Han [2] membawa FXLMS ke mikrokontroler STM32H743 dengan pendekatan *hybrid fixed-filter*. Reduksi sekitar 15 dB pada 200 Hz tidak berubah, tetapi waktu konvergensi pada 300 Hz berkurang dari 10 s menjadi 5 s. Kedua penelitian ini menegaskan bahwa FXLMS sensitif terhadap pemilihan μ dan cenderung lambat berkonvergensi, tetapi keduanya belum membandingkan FXLMS dengan varian ternormalisasinya dan tidak melaporkan

penggunaan sumber daya perangkat keras.

Dari sisi implementasi perangkat keras, Chen et al. [1] merealisasikan FXLMS dengan arsitektur *feedback* dalam bentuk rangkaian elektronik untuk derau impulsif frekuensi rendah dan memperoleh reduksi maksimum 21,8 dB. Penelitian tersebut juga menunjukkan bahwa rentang parameter pada simulasi, yaitu orde 30–60 dan μ antara 10^{-3} dan 10^{-2}, harus disesuaikan kembali menjadi orde 30 dan $\mu = 10^{-3}$ ketika dipindahkan ke *hardware*. Sementara itu, Voltolini et al. menjalankan FXNLMS secara *real-time* pada DSP ADAU1466Z untuk derau struktural pembangkit kogenerasi dan memperoleh hasil yang identik dengan simulasi *offline*, dengan reduksi 3,6–4,0 dB(A). Teknik *downsampling* 16× yang digunakan untuk menekan beban komputasi pada penelitian tersebut relevan bagi implementasi FPGA. Meskipun demikian, kedua penelitian ini tidak menyandingkan FXLMS dan FXNLMS secara langsung. Penelitian pertama tidak menggunakan FXNLMS, penelitian kedua tidak menggunakan FXLMS sebagai pembanding, dan keduanya tidak menyertakan analisis sumber daya digital.

Penelitian yang paling dekat dengan topik ini adalah Cesur et al. [4], yang menerapkan FXNLMS dengan *variable step-size* dan pemodelan jalur sekunder *online* pada kabin kendaraan listrik. Penelitian tersebut memperoleh reduksi rata-rata 5,07 dB, dengan reduksi terbesar 8,28 dB pada 250 Hz, dan reduksi yang turun menjadi 2,4–3,6 dB di atas 1 kHz. FXNLMS dinyatakan lebih cepat berkonvergensi dan menghasilkan reduksi yang lebih besar dibandingkan FXLMS standar, tetapi perbandingan tersebut hanya disampaikan secara kualitatif tanpa data kuantitatif, tidak diimplementasikan pada FPGA, dan tidak disertai laporan penggunaan sumber daya.

Berdasarkan Tabel 2.1, terdapat tiga hal yang belum terjawab oleh penelitian terdahulu. Pertama, sebagian besar penelitian hanya mengevaluasi satu algoritma, yaitu FXLMS [1, 2, 3] atau FXNLMS saja, sehingga perbandingan langsung kedua algoritma pada kondisi uji yang sama masih jarang dilakukan. Kedua, perbandingan FXLMS dan FXNLMS yang tersedia belum didukung data kuantitatif [4]. Ketiga, belum ada penelitian yang melaporkan biaya komputasi kedua algoritma dalam bentuk utilisasi sumber daya FPGA; data efisiensi perangkat keras FPGA yang tersedia berasal dari algoritma adaptif lain, seperti *Signed-Error* LMS [5]. Penelitian ini mengisi celah tersebut dengan membandingkan FXLMS dan FXNLMS secara kuantitatif pada sistem ANC *headphone feedforward* SISO, baik dari sisi kinerja peredaman, karakteristik konvergensi, maupun utilisasi sumber daya pada FPGA.

2.2 Dasar Teori

2.2.1 *Active Noise Cancellation* (ANC)

Active Noise Cancellation (ANC) bekerja dengan membangkitkan gelombang suara dari sumber sekunder yang memiliki amplitudo sama dan fasa berlawanan terhadap sumber suara primer, sehingga keduanya saling meniadakan melalui interferensi destruktif [6]. Gelombang *anti-noise* yang berbeda fasa 180° terhadap *noise* akan meredam *noise* tersebut secara efektif [7]. Kebisingan frekuensi rendah sulit dikendalikan dengan solusi peredaman pasif konvensional [1, 7].

Prinsip tersebut dapat dinyatakan melalui superposisi dua gelombang tekanan. Gelombang primer dan gelombang sekunder dinyatakan sebagai [6]:

$$p_1 = A\cos(\omega t - \varphi) \tag{2.1}$$

$$p_2 = \beta A \cos(\omega t - \varphi + \alpha) \tag{2.2}$$

dengan:

- p_1 : tekanan akustik gelombang primer (*noise*),
- p_2 : tekanan akustik gelombang sekunder (*anti-noise*),
- A : amplitudo gelombang primer,
- ω : frekuensi sudut gelombang (rad/s),
- t : waktu (s),
- φ : fasa awal gelombang primer (rad),
- β : perbandingan amplitudo gelombang sekunder terhadap gelombang primer,
- α : selisih fasa gelombang sekunder terhadap gelombang primer (rad).

Besar peredaman energi akibat superposisi kedua gelombang adalah [6]:

$$\Delta = 10 \lg \frac{E_1}{E_2} = -10 \lg \left(1 + 2\beta \cos \alpha + \beta^2\right) \tag{2.3}$$

dengan:

- Δ : besar peredaman energi (dB),
- E_1 : energi gelombang primer sebelum superposisi,
- E_2 : energi hasil superposisi gelombang primer dan gelombang sekunder,
- lg : logaritma basis 10,
- β, α : perbandingan amplitudo dan selisih fasa seperti pada Persamaan (2.2).

Pada sistem ANC digital, sinyal residual yang terukur pada mikrofon *error* merupakan jumlah dari *noise* primer dan keluaran filter kontrol setelah melewati jalur sekunder [6, 8]:

$$e(n) = d(n) + \sum_{i=0}^{M-1} s_i \, y(n - i) \tag{2.4}$$

dengan:

- n : indeks sampel waktu diskrit,
- $e(n)$: sinyal residual (*error*) yang terukur pada mikrofon *error*,
- $d(n)$: *noise* primer pada posisi mikrofon *error*,
- s_i : koefisien ke-i respons impuls jalur sekunder,
- M : panjang respons impuls jalur sekunder,
- $y(n - i)$: keluaran filter kontrol pada sampel ke-$(n - i)$.

Berdasarkan sumber sinyal yang digunakan untuk membentuk sinyal kontrol, sistem ANC dibedakan menjadi beberapa topologi. Pada sistem ANC *feedforward*, sinyal referensi diperoleh dari sumber *noise*. Pada sistem ANC *feedback*, sinyal kontrol diperoleh hanya dengan memfilter sinyal *error* [6], sehingga arsitektur *feedback* tidak memiliki mikrofon referensi tambahan untuk mencuplik sinyal eksternal [1]. Selain kedua topologi tersebut, terdapat pula topologi *hybrid* [1, 6].

Sistem ANC *feedforward* pada *headset* tersusun atas mikrofon referensi eksternal, mikrofon *error* internal, dan sumber sekunder di dalam *earmuff*. Algoritma *filtered-x least mean*

square (FXLMS) digunakan untuk mengadaptasi filter digital yang menggerakkan sumber sekunder tersebut [9].

Sistem ANC *feedforward* memiliki kendala kausalitas, yaitu *delay* jalur primer harus lebih besar daripada total *delay* jalur sekunder. Jika kendala ini tidak terpenuhi, *bandwidth* peredaman menyempit dan peredaman maksimum menurun [9]. Pada perangkat berukuran kecil, *delay* akustik antara mikrofon referensi dan mikrofon *error* menjadi lebih pendek sehingga jendela waktu pemrosesan yang tersedia semakin terbatas. Oleh karena itu, prosesor harus menyelesaikan pencuplikan sinyal referensi, pemrosesan, dan pembangkitan *anti-noise* dengan cepat [7].

2.2.2 Filter Adaptif dan Algoritma LMS

Pada sistem ANC, filter adaptif dapat berbentuk filter *Finite Impulse Response* (FIR), yaitu filter dengan struktur *all-zero* [1]. Keluaran filter FIR adaptif dengan N koefisien dinyatakan sebagai [6]:

$$y(n) = \mathbf{W}^T(n)\,\mathbf{X}(n) = \sum_{i=0}^{N-1} w_i(n)\,x(n-i) \tag{2.5}$$

dengan:

- $y(n)$: keluaran filter adaptif pada sampel ke-n,
- $\mathbf{W}(n) = [w_0(n), w_1(n), \ldots, w_{N-1}(n)]^T$: vektor koefisien filter pada sampel ke-n,
- $w_i(n)$: koefisien ke-i filter pada sampel ke-n,
- $\mathbf{X}(n) = [x(n), x(n-1), \ldots, x(n-N+1)]^T$: vektor N sampel masukan terbaru,
- $x(n-i)$: sampel sinyal masukan pada sampel ke-$(n-i)$,
- N : jumlah koefisien (panjang) filter,
- $(\cdot)^T$: operasi *transpose*.

Galat antara sinyal yang diinginkan dan keluaran filter adalah [6]:

$$e(n) = d(n) - y(n) \tag{2.6}$$

dengan:

- $e(n)$: sinyal galat pada sampel ke-n,
- $d(n)$: sinyal yang diinginkan (*desired signal*) pada sampel ke-n,
- $y(n)$: keluaran filter adaptif pada Persamaan (2.5).

Tujuan adaptasi adalah meminimalkan fungsi biaya *Mean Square Error* (MSE) [6]:

$$J(n) = E\left[e^2(n)\right] \tag{2.7}$$

dengan:

- $J(n)$: fungsi biaya MSE pada sampel ke-n,
- $E[\cdot]$: operator nilai harapan (*expected value*),
- $e(n)$: sinyal galat pada Persamaan (2.6).

Metode *steepest descent* memperbarui koefisien filter berlawanan arah dengan gradien

fungsi biaya [6]:

$$\mathbf{W}(n+1) = \mathbf{W}(n) - \mu\,\nabla(n) \tag{2.8}$$

dengan:

- $\mathbf{W}(n+1)$: vektor koefisien filter hasil pembaruan untuk sampel berikutnya,
- μ : *step-size*, yaitu konstanta positif yang mengatur besar langkah pembaruan,
- $\nabla(n)$: gradien fungsi biaya $J(n)$ terhadap vektor koefisien $\mathbf{W}(n)$.

Algoritma *Least Mean Square* (LMS) menggunakan estimasi gradien sesaat $\hat{\nabla} = -2e(n)\mathbf{X}(n)$, yaitu gradien dari $e^2(n)$ sebagai pengganti nilai harapan $E[e^2(n)]$, sehingga persamaan pembaruan koefisien LMS menjadi [6]:

$$\mathbf{W}(n+1) = \mathbf{W}(n) + 2\mu\,e(n)\,\mathbf{X}(n) \tag{2.9}$$

dengan:

- $\hat{\nabla}$: estimasi gradien sesaat,
- μ : *step-size*,
- $e(n)$: sinyal galat pada Persamaan (2.6),
- $\mathbf{X}(n)$: vektor sampel masukan pada Persamaan (2.5).

Syarat konvergensi algoritma LMS dinyatakan sebagai [6]:

$$0 < \mu < \frac{1}{\mathrm{tr}[\mathbf{R}]} \tag{2.10}$$

dengan:

- $\mathbf{R} = E[\mathbf{X}(n)\mathbf{X}^T(n)]$: matriks autokorelasi sinyal masukan,
- $\mathrm{tr}[\cdot]$: *trace* matriks, yaitu jumlah elemen diagonal utama matriks.

Step-size menentukan kecepatan konvergensi dan kestabilan sistem [1, 6]. *Step-size* yang kecil menghasilkan konvergensi yang lambat, tetapi menghasilkan residual *error* kondisi tunak yang lebih rendah [10]. Hal ini menunjukkan adanya *trade-off* antara laju konvergensi dan kinerja kondisi tunak [8]. Sebaliknya, *step-size* yang terlalu besar dapat membuat sistem tidak stabil [1].

2.2.3 Algoritma FXLMS

Algoritma yang paling banyak digunakan pada aplikasi ANC adalah *filtered-x least mean square* (FXLMS) [11]. Pada sistem ANC, sinyal kontrol tidak langsung bertemu dengan *noise* pada mikrofon *error*, tetapi melewati jalur sekunder, yaitu respons impuls dari *speaker* kontrol ke mikrofon *error* [1, 8]. FXLMS memperhitungkan latensi akustik pada respons antara sumber sekunder dan sensor *error* tersebut [7].

Pada FXLMS, sinyal referensi difilter terlebih dahulu menggunakan estimasi jalur sekunder sehingga diperoleh sinyal referensi terfilter [6, 8]:

$$x_f(n) = \sum_{i=0}^{M-1} \hat{s}_i \, x(n-i) \tag{2.11}$$

$$\mathbf{X}_f(n) = \Big[x_f(n), x_f(n-1), \ldots, x_f(n-N+1)\Big]^T \tag{2.12}$$

dengan:

- $x_f(n)$: sinyal referensi terfilter (*filtered-x*) pada sampel ke-n,
- $\hat{s}_i$: koefisien ke-i estimasi respons impuls jalur sekunder,
- M : panjang estimasi respons impuls jalur sekunder,
- $x(n-i)$: sampel sinyal referensi dari mikrofon referensi pada sampel ke-$(n-i)$,
- $\mathbf{X}_f(n)$: vektor N sampel sinyal referensi terfilter terbaru,
- N : panjang filter adaptif.

Dengan sinyal *error* yang didefinisikan sebagai jumlah *noise* primer dan keluaran jalur sekunder seperti pada Persamaan (2.4), pembaruan koefisien FXLMS dinyatakan sebagai [6]:

$$\mathbf{W}(n+1) = \mathbf{W}(n) - 2\mu \, e(n) \, \mathbf{X}_f(n) \tag{2.13}$$

dengan:

- $\mathbf{W}(n)$: vektor koefisien filter adaptif pada sampel ke-n,
- μ : *step-size* yang bernilai tetap,
- $e(n)$: sinyal *error* pada Persamaan (2.4),
- $\mathbf{X}_f(n)$: vektor sinyal referensi terfilter pada Persamaan (2.12).

Tanda negatif pada Persamaan (2.13) mengikuti definisi sinyal *error* pada Persamaan (2.4) [6].

2.2.4 Algoritma FXNLMS

Algoritma *filtered-x normalized least mean square* (FXNLMS) banyak digunakan pada sistem ANC [12]. Pada FXNLMS, *step-size* dinormalisasi terhadap energi vektor sinyal referensi terfilter [8, 10]. Dengan konvensi sinyal *error* yang sama dengan Persamaan (2.4), pembaruan koefisien FXNLMS dinyatakan sebagai [8]:

$$\mathbf{W}(n+1) = \mathbf{W}(n) - \mu \, e(n) \, \frac{\mathbf{X}_f(n)}{\varepsilon + \left\|\mathbf{X}_f(n)\right\|^2} \tag{2.14}$$

dengan:

- μ : *step-size* sebelum dinormalisasi,
- ε : konstanta positif kecil ($\varepsilon > 0$) yang mencegah terjadinya pembagian dengan nol,
- $\left\|\mathbf{X}_f(n)\right\|^2$: kuadrat norma Euclidean vektor sinyal referensi terfilter, yaitu jumlah kuadrat seluruh elemen $\mathbf{X}_f(n)$,
- $\mathbf{W}(n)$, $e(n)$, $\mathbf{X}_f(n)$: sama seperti pada Persamaan (2.13).

Pembaruan FXLMS ternormalisasi disebutkan lebih disukai pada kondisi yang berubah terhadap waktu [8]. Namun, untuk sinyal masukan siklostasioner, kinerja peredaman FXNLMS

dipengaruhi oleh daya periodik sinyal masukan sehingga menghasilkan *steady-state mean square deviation* (MSD) yang lebih besar dibandingkan FXLMS [12].

Pada algoritma NLMS, faktor normalisasi juga dapat dihitung dari estimasi daya sinyal masukan yang diperbarui secara rekursif [13]:

$$r(n) = (1 - \alpha)\, r(n-1) + \alpha\, x^2(n) \tag{2.15}$$

dengan:

- $r(n)$: estimasi daya sinyal masukan pada sampel ke-n,
- $r(n-1)$: estimasi daya sinyal masukan pada sampel sebelumnya,
- α : faktor peluruhan (*forgetting factor*) yang dapat disesuaikan dengan tingkat ketidakstasioneran sinyal masukan,
- $x(n)$: sampel sinyal masukan pada sampel ke-n.

Estimasi daya tersebut digunakan pada penguatan adaptasi [13]:

$$\mathbf{g}(n) = \frac{\mu\, \mathbf{X}(n)}{c_0 + N\, r(n)} \tag{2.16}$$

dengan:

- $\mathbf{g}(n)$: vektor penguatan adaptasi yang menggantikan suku $\mu\, \mathbf{X}(n)$ pada pembaruan koefisien,
- c_0 : konstanta positif kecil untuk menghindari pembagian dengan nol ketika sinyal masukan tidak ada,
- N : panjang filter adaptif,
- $r(n)$: estimasi daya sinyal masukan pada Persamaan (2.15),
- μ, $\mathbf{X}(n)$: *step-size* dan vektor sampel masukan.

Dari sisi komputasi, setiap iterasi FXNLMS membutuhkan $3L + M + 1$ operasi perkalian, satu operasi pembagian, dan $3L + M - 3$ operasi penjumlahan, dengan L adalah panjang vektor koefisien filter adaptif (setara dengan N pada persamaan di atas) dan M adalah panjang respons impuls jalur sekunder [10]. Perbandingan kedua algoritma berdasarkan persamaan dan literatur di atas dirangkum pada Tabel 2.2.

Tabel 2.2: Perbandingan FXLMS dan FXNLMS

Aspek	**FXLMS**	**FXNLMS**
Pembaruan koefisien	Persamaan (2.13) [6]	Persamaan (2.14) [8]
Step-size	Tetap, μ	$\mu/(\varepsilon + \|\|\mathbf{X}_f(n)\|\|^2)$, mengikuti energi sinyal referensi terfilter
Operasi pembagian per iterasi	Tidak ada	Satu [10]
Catatan dari literatur	Algoritma yang paling banyak digunakan pada ANC [11]	Lebih disukai pada kondisi yang berubah terhadap waktu [8]; MSD tunak lebih besar daripada FXLMS untuk masukan siklostasioner [12]

2.2.5 *Field-Programmable Gate Array* (FPGA)

Field-Programmable Gate Array (FPGA) adalah perangkat terfabrikasi yang dapat diprogram secara elektris untuk menjadi hampir semua jenis rangkaian atau sistem digital. FPGA tersusun atas larik blok logika yang dapat diprogram, yang dikelilingi oleh jaringan *routing* dan blok I/O yang juga dapat diprogram [14]. Pada FPGA berbasis SRAM, konfigurasi bersifat *volatile* sehingga harus dimuat ulang setiap kali FPGA dinyalakan [14, 15].

Sumber daya logika FPGA dibentuk oleh *lookup table* (LUT) dan *flip-flop*. Pada FPGA 7 series, empat LUT 6 masukan beserta delapan *flip-flop* dan logika *carry* membentuk satu *slice* [15]. Selain itu, FPGA dilengkapi blok khusus (*hard block*) seperti memori dan pengali, yang umumnya lebih unggul dalam luas area, kecepatan, dan konsumsi daya dibandingkan fungsi yang sama pada logika serbaguna [14]. Pada FPGA 7 series, blok khusus tersebut meliputi *block RAM dual-port* 36 Kb dan DSP *slice* yang terdiri atas pengali *two's complement* 25×18 bit dan akumulator 48 bit dengan kemampuan *pipelining* [15].

Pemrosesan algoritma FXLMS ditandai oleh banyaknya operasi *multiply-accumulate* (MAC) yang harus diselesaikan dalam setiap periode sampel. Daya komputasi menjadi kriteria terpenting untuk memungkinkan filter berorde tinggi yang dikombinasikan dengan frekuensi *sampling* yang tinggi [16]. Kesesuaian suatu teknologi implementasi untuk ANC terutama ditentukan oleh daya pemrosesan, disertai *delay* pemrosesan dan efisiensi energi, dan kinerja ANC juga dapat meningkat dengan tambahan daya komputasi [16].

Selain daya komputasi, sistem ANC membutuhkan *delay* pemrosesan yang kecil karena waktu yang tersedia untuk meredam *noise* secara aktif terbatas [16]. Keterbatasan waktu ini berkaitan langsung dengan kendala kausalitas ANC *feedforward* pada Subbab 2.2.1, yaitu total *delay* jalur sekunder harus lebih kecil daripada *delay* jalur primer [9]. Pada perangkat berukuran kecil seperti *headphone*, *delay* akustik antara mikrofon referensi dan mikrofon *error* lebih pendek, sehingga jendela waktu pemrosesan semakin terbatas [7].

FPGA memiliki beberapa karakteristik yang menjawab kebutuhan tersebut. Pertama, pada logika FPGA, sampel audio dari antarmuka A/D dapat langsung diproses oleh gerbang dan

register, sehingga *overhead* manajemen pada prosesor serbaguna atau sistem operasi dapat dihindari dan *delay* pemrosesan menjadi lebih kecil. Kedua, FPGA lebih dipilih untuk algoritma aritmetika sederhana yang sesuai untuk dieksekusi secara paralel. Ketiga, jalur data (*datapath*) pada FPGA dapat disesuaikan pada tingkat *register-transfer level* (RTL), yaitu tingkat abstraksi yang lebih rendah daripada urutan instruksi perangkat lunak pada prosesor DSP, sehingga optimasi dapat dilakukan lebih jauh [16]. Keempat, operasi perkalian dan akumulasi yang mendominasi FXLMS dapat dipetakan ke DSP *slice* yang memang dirancang sebagai pengali dan akumulator khusus dengan dukungan *pipelining* [15], sehingga memperoleh keunggulan blok khusus dalam luas area, kecepatan, dan konsumsi daya [14].

Pada prosesor DSP, utilisasi perangkat keras aritmetika per siklus *clock* dapat menurun secara signifikan ketika pembaruan koefisien harus ditangani bersamaan dengan proses konvolusi [16]. Pada FXLMS dan FXNLMS, kedua proses tersebut dilakukan pada setiap sampel, yaitu perhitungan keluaran filter pada Persamaan (2.5) dan pembaruan koefisien pada Persamaan (2.13) dan (2.14). Klemd juga mengutip hasil penelitian yang menunjukkan bahwa rancangan FPGA yang diparalelkan pada frekuensi 100 MHz dapat melampaui prosesor DSP 1,5 GHz dengan percepatan 12,3 kali dan konsumsi daya sekitar 2,4 kali lebih rendah pada teknologi proses yang sama [16].

Keunggulan tersebut terlihat pada hasil penelitian Klemd. Implementasi FXLMS *feedback* pada FPGA mencapai *delay* pemrosesan yang pendek dan tidak bergantung pada panjang filter. Pada konfigurasi paling berat, yaitu 11 sensor *error*, 11 sumber sekunder, orde filter 2049, dan frekuensi *sampling* 48 kHz, *delay* yang dicapai jauh di bawah 1 μs dengan tambahan peredaman hingga 12 dB dibandingkan implementasi perangkat lunak acuan [16]. Arsitektur FXLMS pada FPGA juga dapat diparameterisasi untuk diarahkan pada kinerja tinggi maupun efisiensi sumber daya [11].

BAB III
METODOLOGI

3.1 Metode Penelitian

Penelitian ini menggunakan metode eksperimen kuantitatif untuk membandingkan kinerja dan efisiensi komputasi algoritma FXLMS dan FXNLMS pada sistem ANC *headphone*. Alur penelitian dimulai dari perancangan model sistem ANC *feedforward* SISO dan estimasi jalur sekunder pada MATLAB atau Python. Kedua algoritma kemudian disimulasikan menggunakan rekaman bising lingkungan nyata dari repositori DEMAND, QUT-NOISE, dan MUSAN untuk memperoleh besar peredaman, kecepatan konvergensi, dan *steady-state error*. Setelah hasil simulasi dinyatakan sesuai, kedua algoritma dirancang menggunakan HDL dengan aritmetika *fixed-point* dan disintesis pada FPGA Artix-7 XC7A35T untuk memperoleh utilisasi sumber daya. Rancangan tersebut kemudian diverifikasi pada prototipe ANC *headphone*. Hasil akhir dibandingkan antara FXLMS dengan *step-size* tetap dan FXNLMS dengan *step-size* ternormalisasi.

Metode eksperimen kuantitatif dipilih karena ketiga rumusan masalah menuntut perbandingan numerik kedua algoritma pada kondisi uji yang sama. Kedua algoritma dibangun dengan filter adaptif, estimasi jalur sekunder, sinyal uji, dan parameter yang identik. Dengan demikian, perbedaan hasil hanya berasal dari mekanisme pembaruan koefisien. Seluruh pengujian dibatasi pada sistem ANC *feedforward single-channel* (SISO) yang merepresentasikan satu sisi *earcup* pada *headphone*.

3.2 Alat dan Bahan Penelitian

Alat dan bahan penelitian ditunjukkan pada Tabel 3.1. Alat penelitian mencakup perangkat keras prototipe ANC, perangkat ukur dan pendukung, serta perangkat lunak simulasi, sintesis, dan analisis data. Bahan penelitian berupa rekaman bising lingkungan nyata.

Tabel 3.1: Alat dan bahan penelitian

Alat/Bahan	**Spesifikasi**	**Fungsi**
Board FPGA	Digilent Arty A7-35T, Xilinx Artix-7 XC7A35T, 20.800 LUT, 41.600 FF, 90 DSP48E1, 100 MHz	Platform implementasi algoritma FXLMS dan FXNLMS serta sumber catu 3,3 V bagi mikrofon, DAC, dan penguat
Mikrofon referensi	INMP441, MEMS, antarmuka I2S, 24-bit, *omnidirectional*, 1 unit	Akuisisi sinyal kebisingan di luar *earcup*
Mikrofon *error*	INMP441, MEMS, antarmuka I2S, 24-bit, *omnidirectional*, 1 unit	Akuisisi sinyal residual di dalam *earcup*, dekat telinga
DAC audio	PCM5102A, antarmuka I2S, 32-bit, 112 dB SNR	Konversi sinyal *anti-noise* digital menjadi analog

Tabel 3.1: Alat dan bahan penelitian (lanjutan)

Alat/Bahan	**Spesifikasi**	**Fungsi**
Penguat *headphone*	TPA6132A2, stereo, *class*-G, 2,3–5,5 V	Penguatan sinyal analog untuk menggerakkan *speaker*
Speaker driver	Dinamis 40 mm, 32 Ω, 2 unit (1 aktif, 1 cadangan)	Pembangkit sinyal *anti-noise*
Mikrofon ukur	MIC-L20 *Clip On Microphone* USB	Pengukuran independen tingkat peredaman pada prototipe
Sumber kebisingan	*Speaker* aktif/*studio monitor*	Pemutar rekaman bising lingkungan nyata pada prototipe
Komputer	Windows 11/Linux	Menjalankan perangkat lunak simulasi, sintesis, dan analisis data
Vivado Design Suite	Perangkat lunak sintesis, implementasi, dan simulasi HDL untuk FPGA Xilinx	Verifikasi *testbench*, sintesis HDL, serta pengambilan laporan utilisasi LUT, *flip-flop*, BRAM, dan DSP *slice*
MATLAB/Python	Perangkat lunak simulasi dan analisis numerik	Simulasi sistem ANC, estimasi jalur sekunder, dan perhitungan metrik kinerja
Rekaman bising lingkungan nyata	DEMAND, QUT-NOISE, dan MUSAN (Tabel 3.2)	Sinyal kebisingan untuk simulasi, estimasi jalur sekunder, dan pengujian prototipe

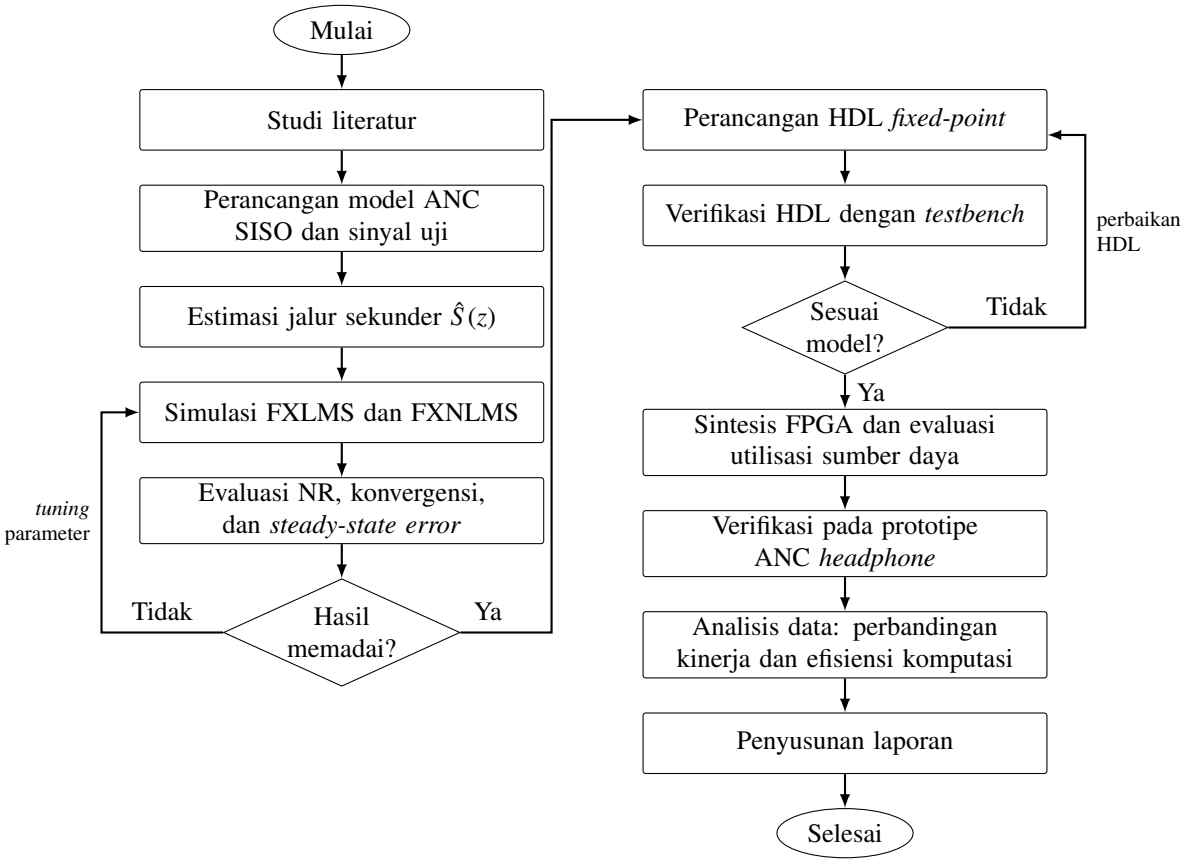
Seluruh sinyal kebisingan pada penelitian ini diambil dari tiga repositori *open-source* yang dirangkum pada Tabel 3.2. Ketiga repositori dipilih karena menyediakan bising dari lingkungan yang beragam, seperti kendaraan, jalan raya, kantor, dan ruang publik, dengan lisensi terbuka.

Tabel 3.2: Repositori sinyal bising lingkungan nyata

Dataset	**Repositori**	**Isi**	**Format**	**Lisensi**
DEMAND [17]	Zenodo (DOI 10.5281/ zenodo.1227121)	17 lingkungan, antara lain kendaraan (TBUS, TCAR, TMETRO), jalan raya (STRAFFIC), kantor, rumah, dan ruang publik	16 kanal, 48 kHz dan 16 kHz	CC BY 4.0
QUT-NOISE [18]	QUT Research Data Finder	20 sesi di 10 lokasi (CAFE, CAR, HOME, REVERB, STREET), lebih dari 10 jam, setiap sesi minimal 30 menit	Stereo, 48 kHz, 16 bit	CC BY-SA
MUSAN (bagian *noise*) [19]	OpenSLR (SLR17)	929 berkas, sekitar 6 jam; bising teknis dan bising lingkungan seperti mobil *idle*, angin, hujan, dan petir	16 kHz	CC BY 4.0

3.3 Alur Pelaksanaan Penelitian

Alur pelaksanaan penelitian ditunjukkan pada Gambar 3.1. Penelitian diawali dengan studi literatur mengenai ANC, algoritma FXLMS dan FXNLMS, serta implementasi filter adaptif pada FPGA. Tahap berikutnya adalah perancangan model sistem ANC SISO, penyiapan sinyal uji, dan estimasi jalur sekunder $\hat{S}(z)$. Setelah itu, FXLMS dan FXNLMS disimulasikan dan dievaluasi berdasarkan besar peredaman, kecepatan konvergensi, dan *steady-state error*.



Gambar 3.1: Diagram alir pelaksanaan penelitian

Jika hasil simulasi belum memadai, parameter simulasi seperti *step-size* dan panjang filter disetel ulang, kemudian simulasi diulang. Jika sudah memadai, kedua algoritma dirancang dalam HDL *fixed-point* dan diverifikasi melalui simulasi *testbench*.

Jika keluaran HDL belum sesuai dengan model simulasi, rancangan HDL diperbaiki. Jika sudah sesuai, rancangan disintesis pada FPGA untuk memperoleh utilisasi sumber daya, lalu diverifikasi pada prototipe ANC *headphone*. Tahap terakhir adalah analisis data yang membandingkan kinerja dan efisiensi komputasi kedua algoritma, kemudian dilanjutkan dengan penyusunan laporan tugas akhir. Penelitian ini direncanakan berlangsung selama satu semester (6 bulan). Rincian jadwal kegiatan disajikan pada Tabel 3.3.

Tabel 3.3: Jadwal pelaksanaan tugas akhir

No.	Kegiatan	Bulan 1	Bulan 2	Bulan 3	Bulan 4	Bulan 5	Bulan 6
1	Identifikasi masalah						
2	Studi literatur						
3	Perancangan model sistem ANC, sinyal uji, dan estimasi jalur sekunder						
4	Implementasi FXLMS dan FXNLMS pada simulasi						
5	Simulasi dan evaluasi peredaman, konvergensi, dan *steady-state error*						
6	Perancangan HDL dan verifikasi *test-bench*						
7	Sintesis dan evaluasi utilisasi sumber daya						
8	Pengujian prototipe dan pengukuran peredaman						
9	Analisis data: perbandingan kinerja dan efisiensi komputasi						
10	Penyusunan laporan tugas akhir						

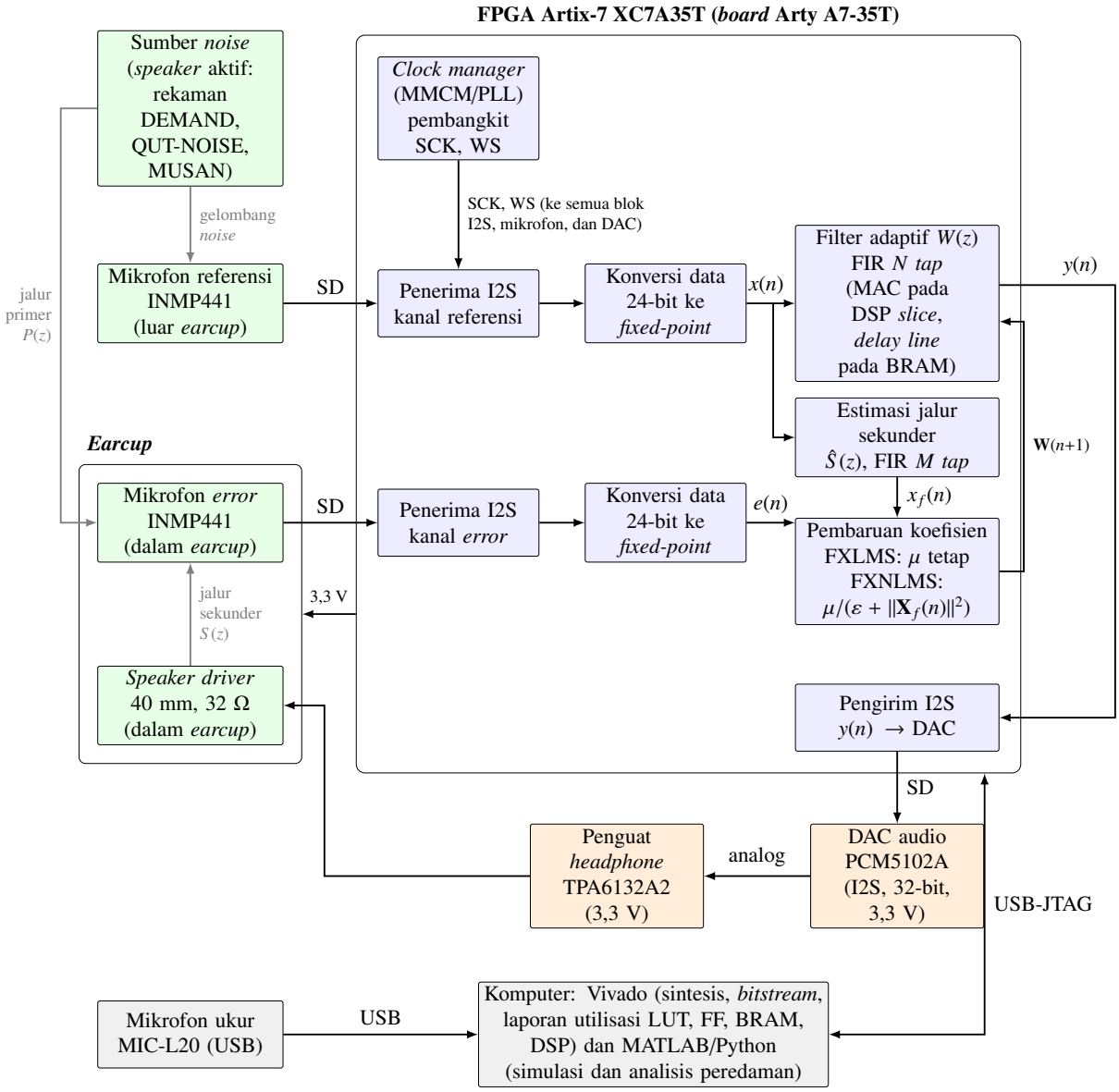
3.4 Perancangan Sistem ANC *Headphone* Berbasis FPGA

Bagian ini menjelaskan rancangan sistem ANC *headphone* yang digunakan untuk membandingkan FXLMS dan FXNLMS. Pembahasan mencakup arsitektur sistem, model simulasi, estimasi jalur sekunder, rancangan algoritma FXLMS dan FXNLMS, serta implementasi pada FPGA.

3.4.1 Arsitektur Sistem

Sistem dirancang sebagai ANC *feedforward* SISO untuk satu sisi *earcup headphone*. Algoritma kontrol diimplementasikan pada FPGA Artix-7 XC7A35T (*board* Arty A7-35T). FPGA menerima sinyal dari dua mikrofon, menghitung sinyal *anti-noise*, dan mengirimkan sinyal tersebut ke *speaker* melalui rantai analog keluaran.

Diagram blok sistem ditunjukkan pada Gambar 3.2. Diagram tersebut terbagi menjadi empat bagian, yaitu domain akustik, FPGA, rantai analog keluaran, serta perangkat pendukung.



Gambar 3.2: Diagram blok sistem ANC *headphone* berbasis FPGA

Pada domain akustik, sumber *noise* berupa *speaker* aktif memutar rekaman bising dari repositori pada Tabel 3.2. Gelombang *noise* ditangkap oleh mikrofon referensi di luar *earcup* sebagai sinyal referensi. Gelombang yang sama merambat melalui jalur primer $P(z)$ menuju mikrofon *error* di dalam *earcup*. Mikrofon *error* menangkap sinyal residual, yaitu gabungan *noise* dari jalur primer dan *anti-noise* dari *speaker driver* yang merambat melalui jalur sekunder $S(z)$. Kedua mikrofon dicatu langsung oleh jalur 3,3 V *board* Arty A7-35T melalui pin VCC Pmod.

Pada FPGA, *clock manager* membangkitkan sinyal SCK dan WS sehingga FPGA berperan sebagai *master* bagi kedua mikrofon dan DAC. Data serial (SD) dari kedua mikrofon diterima oleh blok penerima I2S, kemudian dikonversi dari format 24-bit menjadi format *fixed-point*. Hasil konversi tersebut adalah sinyal referensi $x(n)$ dan sinyal *error* $e(n)$. Sinyal $x(n)$ diproses oleh filter adaptif $W(z)$ untuk menghasilkan sinyal *anti-noise* $y(n)$. Secara paralel, $x(n)$ difilter oleh $\hat{S}(z)$ untuk menghasilkan sinyal referensi terfilter $x_f(n)$. Blok pembaruan koefisien meng-

gunakan $x_f(n)$ dan $e(n)$ untuk menghitung koefisien baru $\mathbf{W}(n+1)$.

Sinyal $y(n)$ dikirim melalui blok pengirim I2S ke DAC PCM5102A. Sinyal analog keluaran DAC dikuatkan oleh penguat TPA6132A2 untuk menggerakkan *speaker driver* di dalam *earcup*. DAC dan penguat juga dicatu oleh jalur 3,3 V *board* Arty A7-35T, sedangkan *speaker driver* bersifat pasif dan memperoleh daya dari keluaran penguat. Komputer digunakan untuk sintesis dan pemrograman FPGA melalui USB-JTAG, pengambilan laporan utilisasi sumber daya, serta simulasi dan analisis data. Tingkat peredaman pada prototipe diukur secara independen menggunakan mikrofon ukur MIC-L20 yang terhubung ke komputer.

3.4.2 Model Simulasi Sistem ANC

Sebelum diimplementasikan pada FPGA, sistem ANC dimodelkan pada MATLAB atau Python. Model mengikuti struktur ANC *feedforward* pada Bab 2, yaitu jalur primer $P(z)$, jalur sekunder $S(z)$, estimasi jalur sekunder $\hat{S}(z)$, dan filter adaptif $W(z)$ berstruktur FIR sepanjang *N tap*. Jalur primer dan jalur sekunder dimodelkan sebagai filter FIR, sehingga sinyal residual pada mikrofon *error* mengikuti Persamaan (2.4). Struktur model dibuat sama dengan Gambar 3.2 agar hasil simulasi dapat langsung dibandingkan dengan hasil pada FPGA.

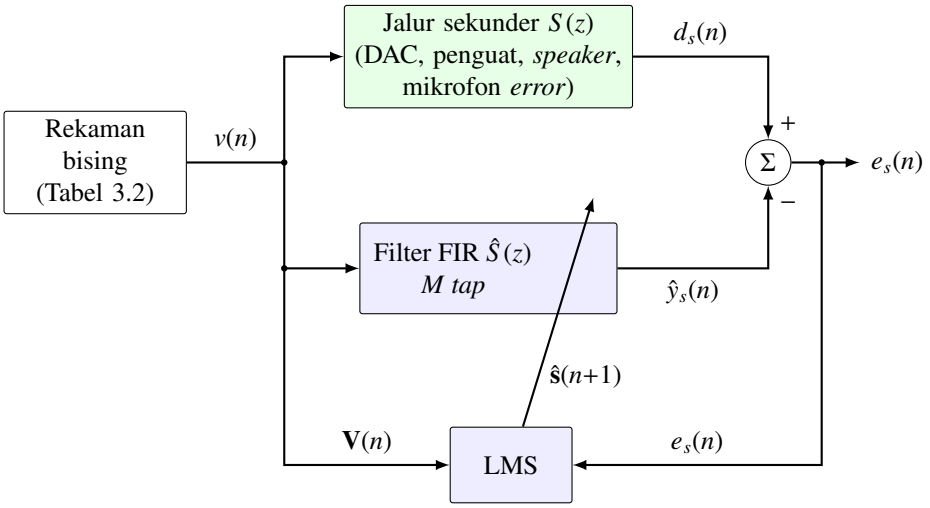
Sinyal uji yang digunakan adalah rekaman bising lingkungan nyata dari repositori DEMAND, QUT-NOISE, dan MUSAN pada Tabel 3.2. Rekaman tersebut digunakan pada pita frekuensi rendah di bawah 2000 Hz.

Kedua algoritma diuji pada kondisi yang sama, yaitu sinyal uji, model jalur primer dan sekunder, panjang filter, frekuensi *sampling*, dan kondisi awal koefisien filter. Nilai *step-size* pada FXLMS ditentukan melalui penelusuran beberapa nilai (*tuning*). Cara ini dipilih agar FXLMS dibandingkan pada kondisi terbaiknya, bukan pada nilai yang kebetulan merugikan.

3.4.3 Estimasi Jalur Sekunder

FXLMS dan FXNLMS sama-sama membutuhkan estimasi jalur sekunder $\hat{S}(z)$ untuk menghasilkan sinyal referensi terfilter $x_f(n)$ pada Persamaan (2.11). Oleh karena itu, jalur sekunder diestimasi terlebih dahulu sebelum sistem ANC dijalankan (*offline*). Pada tahap ini, filter adaptif $W(z)$ belum dijalankan, dan *speaker* hanya digunakan untuk membangkitkan sinyal identifikasi.

Skema estimasi jalur sekunder ditunjukkan pada Gambar 3.3. Sinyal identifikasi $v(n)$ berupa rekaman bising dari repositori pada Tabel 3.2 diberikan ke *speaker* melalui rantai keluaran yang sama dengan sinyal *anti-noise*, yaitu DAC, penguat, dan *speaker driver*. Sinyal yang terukur pada mikrofon *error* digunakan sebagai sinyal yang diinginkan $d_s(n)$. Sinyal $v(n)$ yang sama digunakan sebagai masukan filter FIR $\hat{S}(z)$ sepanjang *M tap*.



Gambar 3.3: Skema estimasi jalur sekunder secara *offline*

Keluaran $\hat{S}(z)$ dikurangkan dari $d_s(n)$ sehingga diperoleh galat estimasi $e_s(n)$. Galat $e_s(n)$ dan vektor masukan $\mathbf{V}(n) = [v(n), v(n-1), \ldots, v(n-M+1)]^T$ diberikan ke blok LMS. Blok LMS menghitung koefisien baru $\hat{\mathbf{s}}(n+1)$ mengikuti Persamaan (2.9):

$$\hat{\mathbf{s}}(n+1) = \hat{\mathbf{s}}(n) + 2\mu_s\, e_s(n)\, \mathbf{V}(n) \tag{3.1}$$

dengan $\hat{\mathbf{s}}(n) = [\hat{s}_0(n), \hat{s}_1(n), \ldots, \hat{s}_{M-1}(n)]^T$ sebagai vektor koefisien $\hat{S}(z)$ dan μ_s sebagai *step-size* estimasi. Pada Persamaan (3.1), $e_s(n)$ berperan sebagai sinyal galat dan $\mathbf{V}(n)$ berperan sebagai vektor masukan pada LMS. Koefisien baru tersebut dimuat kembali ke filter FIR $\hat{S}(z)$, yang ditunjukkan oleh panah diagonal dari blok LMS yang menembus blok $\hat{S}(z)$ pada Gambar 3.3. Proses ini diulang hingga $e_s(n)$ mencapai kondisi tunak. Pengukuran dilakukan tanpa sumber *noise* eksternal agar sinyal pada mikrofon *error* hanya berasal dari *speaker*.

Pada tahap simulasi, $\hat{S}(z)$ diperoleh dengan prosedur yang sama terhadap model FIR $S(z)$. Pada prototipe, sinyal mikrofon *error* direkam selama $v(n)$ diputar, kemudian koefisien $\hat{S}(z)$ dihitung dan disimpan pada FPGA sebagai koefisien tetap. Koefisien $\hat{S}(z)$ yang sama digunakan pada FXLMS dan FXNLMS. Dengan demikian, perbedaan hasil kedua algoritma tidak dipengaruhi oleh kualitas estimasi jalur sekunder.

3.4.4 Rancangan Algoritma FXLMS dan FXNLMS

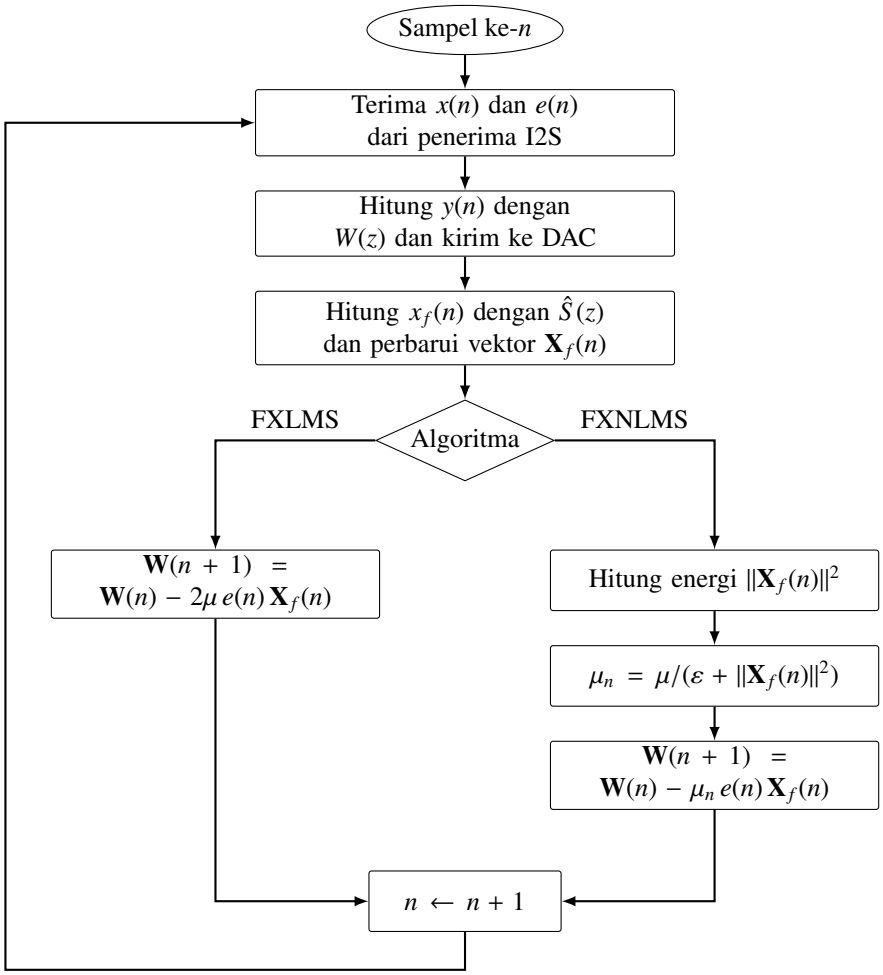
FXLMS dan FXNLMS dirancang dengan struktur yang identik. Keduanya menggunakan filter adaptif $W(z)$ yang sama, estimasi $\hat{S}(z)$ yang sama, dan sinyal referensi terfilter $x_f(n)$ yang sama. Perbedaan kedua algoritma hanya terdapat pada blok pembaruan koefisien. Rancangan awal kedua algoritma dirangkum pada Tabel 3.4.

Tabel 3.4: Rancangan awal algoritma dan dugaan awal

Algoritma	**Bentuk Umum Rancangan**	**Dugaan Awal**
FXLMS	Filter adaptif FIR dengan *step-size* tetap μ. Pembaruan koefisien sesuai Persamaan (2.13): $\mathbf{W}(n+1) = \mathbf{W}(n) - 2\mu\, e(n)\, \mathbf{X}_f(n)$	Operasi lebih sederhana dan hemat sumber daya, tetapi kinerjanya bergantung pada *tuning* μ dan rentan terhadap variasi daya sinyal
FXNLMS	Struktur sama dengan FXLMS, dengan *step-size* dinormalisasi oleh energi sinyal referensi terfilter. Pembaruan koefisien sesuai Persamaan (2.14): $\mathbf{W}(n+1) = \mathbf{W}(n) - \dfrac{\mu\, e(n)\, \mathbf{X}_f(n)}{\varepsilon + \|\mathbf{X}_f(n)\|^2}$	Konvergensi lebih cepat dan lebih tahan terhadap fluktuasi daya sinyal, tetapi memerlukan perhitungan energi dan operasi pembagian tambahan

Pada kedua rancangan, sinyal $e(n)$ berperan sebagai sinyal galat, vektor $\mathbf{X}_f(n)$ berperan sebagai masukan yang menunjukkan arah koreksi tiap koefisien, dan μ berperan sebagai *step-size*. Pada FXLMS, besar langkah pembaruan bernilai tetap. Pada FXNLMS, besar langkah pembaruan dibagi dengan energi $\|\mathbf{X}_f(n)\|^2$ yang dihitung dari jumlah kuadrat elemen vektor $\mathbf{X}_f(n)$. Konstanta ε ditambahkan agar pembagian tetap terdefinisi ketika energi sinyal mendekati nol.

Alur pemrosesan setiap sampel ditunjukkan pada Gambar 3.4. Alur dimulai dari penerimaan $x(n)$ dan $e(n)$. Keluaran filter $y(n)$ dihitung menggunakan Persamaan (2.5), kemudian dikirim ke DAC. Setelah itu, $x_f(n)$ dihitung melalui $\hat{S}(z)$ dan dimasukkan ke vektor $\mathbf{X}_f(n)$. Alur bercabang pada tahap pembaruan koefisien. FXLMS langsung memperbarui $\mathbf{W}(n)$ dengan μ tetap, sedangkan FXNLMS terlebih dahulu menghitung energi $\|\mathbf{X}_f(n)\|^2$ dan *step-size* ternormalisasi.



Gambar 3.4: Alur pemrosesan setiap sampel pada FXLMS dan FXNLMS

Berdasarkan rancangan tersebut, ditetapkan dugaan awal terhadap karakteristik kedua algoritma. FXLMS diduga lebih hemat sumber daya, tetapi kinerjanya bergantung pada pemilihan μ dan rentan terhadap variasi daya sinyal. FXNLMS diduga lebih stabil pada daya sinyal yang berfluktuasi, tetapi membutuhkan perhitungan energi dan operasi pembagian tambahan pada setiap iterasi. Kesimpulan akhir tetap ditentukan berdasarkan hasil eksperimen.

3.4.5 Implementasi pada FPGA

Kedua algoritma dirancang menggunakan HDL dengan aritmetika *fixed-point* dan disintesis menggunakan Vivado pada *board* Arty A7-35T. FPGA diposisikan sebagai platform untuk mengukur biaya komputasi algoritma, bukan sebagai fokus perancangan perangkat keras tingkat rendah. Panjang filter N dan M, lebar bit data, dan frekuensi *clock* dibuat sama pada kedua rancangan. Nilai tersebut dipilih sama agar perbedaan utilisasi sumber daya hanya berasal dari perbedaan algoritma.

Rancangan HDL terdiri atas blok penerima I2S, konversi data, filter FIR $W(z)$, filter FIR $\hat{S}(z)$, pembaruan koefisien, dan pengirim I2S sesuai Gambar 3.2. Blok pembaruan koefisien menjadi satu-satunya blok yang berbeda antara kedua rancangan. Pada FXNLMS, blok tersebut ditambah dengan perhitungan energi $\|\mathbf{X}_f(n)\|^2$ dan satu operasi pembagian. Operasi perkalian dan akumulasi pada filter dipetakan ke DSP *slice*, sedangkan *delay line* sinyal disimpan pada BRAM. Koefisien $\hat{S}(z)$ hasil estimasi *offline* disimpan sebagai koefisien tetap.

3.5 Metode Pengujian dan Analisis Data

Pengujian dilakukan untuk menjawab ketiga rumusan masalah. Seluruh metrik dihitung dengan prosedur yang sama untuk FXLMS dan FXNLMS agar hasilnya dapat dibandingkan secara langsung. Pembahasan mencakup kinerja peredaman, kecepatan konvergensi dan *steady-state error*, verifikasi rancangan HDL dan prototipe, efisiensi komputasi, serta analisis data akhir.

3.5.1 Pengujian Kinerja Peredaman

Kinerja peredaman dinyatakan sebagai besar peredaman kebisingan (*noise reduction*, NR) dalam dB. NR merupakan perbandingan daya *noise* pada mikrofon *error* tanpa ANC terhadap daya sinyal residual setelah ANC bekerja:

$$\mathrm{NR} = 10 \log_{10} \left(\frac{\sum_n d^2(n)}{\sum_n e^2(n)} \right) \tag{3.2}$$

dengan $d(n)$ sebagai *noise* primer pada mikrofon *error* tanpa ANC dan $e(n)$ sebagai sinyal residual setelah ANC bekerja. Penjumlahan dilakukan pada rentang sampel setelah algoritma mencapai kondisi tunak.

Nilai NR dihitung untuk setiap rekaman bising dari repositori DEMAND, QUT-NOISE, dan MUSAN. Hasil tersebut digunakan untuk menjawab rumusan masalah pertama.

3.5.2 Pengujian Kecepatan Konvergensi dan *Steady-State Error*

Perilaku konvergensi diamati melalui kurva pembelajaran (*learning curve*), yaitu nilai MSE terhadap jumlah iterasi. Karena hasil adaptasi bergantung pada potongan rekaman yang digunakan, setiap skenario diulang sebanyak R kali dengan potongan rekaman yang berbeda, kemudian hasilnya dirata-ratakan:

$$\mathrm{MSE}(n) = \frac{1}{R} \sum_{r=1}^{R} e_r^2(n) \tag{3.3}$$

dengan $\mathrm{MSE}(n)$ sebagai nilai MSE rata-rata pada iterasi ke-n dan $e_r(n)$ sebagai sinyal *error* pada iterasi ke-n dari pengulangan ke-r.

Steady-state error dihitung sebagai rata-rata $\mathrm{MSE}(n)$ pada K iterasi terakhir setelah algoritma konvergen. Kecepatan konvergensi dinyatakan sebagai jumlah iterasi yang dibutuhkan hingga $\mathrm{MSE}(n)$ masuk dan tetap berada dalam pita toleransi di sekitar nilai *steady-state error* tersebut.

Pengujian dilakukan pada dua skenario dengan kerangka simulasi pada Subbab 3.4.2. Skenario pertama menggunakan rekaman bising dengan tingkat daya tetap. Skenario kedua menggunakan rekaman bising yang tingkat dayanya diubah secara tiba-tiba selama simulasi untuk memunculkan fluktuasi daya sinyal *broadband*. Skenario kedua dipilih karena *step-size* tetap pada FXLMS dirancang untuk satu tingkat daya sinyal referensi.

Pada kedua skenario, FXLMS diuji dengan beberapa nilai *step-size* untuk menunjukkan hubungan antara kecepatan konvergensi dan kestabilan. FXNLMS diuji dengan konfigurasi yang sama pada setiap skenario. Kurva pembelajaran kedua algoritma dibandingkan sebelum

dan sesudah perubahan daya. Kecepatan konvergensi dan *steady-state error* dari kedua skenario digunakan untuk menjawab rumusan masalah kedua.

3.5.3 Verifikasi Rancangan HDL dan Prototipe

Rancangan HDL diverifikasi menggunakan *testbench* pada simulator Vivado sebelum disintesis. Sinyal uji yang sama dengan simulasi diberikan sebagai masukan $x(n)$ dan $e(n)$. Keluaran $y(n)$ dan koefisien $\mathbf{W}(n)$ dari HDL kemudian dibandingkan dengan keluaran model simulasi. Jika hasilnya belum sesuai, rancangan HDL diperbaiki sebelum dilanjutkan ke tahap sintesis.

Setelah disintesis, rancangan diuji pada prototipe ANC *headphone*. Rekaman bising dari repositori pada Tabel 3.2 diputar melalui *speaker* aktif sebagai sumber kebisingan. Peredaman diukur sebagai selisih tingkat tekanan suara pada mikrofon ukur antara kondisi ANC tidak aktif dan kondisi ANC aktif. Pengujian prototipe berfungsi sebagai verifikasi implementasi, sedangkan evaluasi kinerja algoritma tetap bertumpu pada hasil simulasi sesuai batasan penelitian.

3.5.4 Evaluasi Efisiensi Komputasi

Evaluasi ini dilakukan untuk menjawab rumusan masalah ketiga. Efisiensi komputasi dianalisis dari dua sisi. Pertama, jumlah operasi perkalian, penjumlahan, dan pembagian per iterasi kedua algoritma dibandingkan secara analitis berdasarkan Subbab 2.2.4 dan Tabel 2.2. Kedua, utilisasi sumber daya hasil sintesis dan implementasi Vivado dibandingkan untuk setiap jenis sumber daya:

$$U = \frac{N_{\text{terpakai}}}{N_{\text{tersedia}}} \times 100\% \tag{3.4}$$

dengan U sebagai persentase utilisasi suatu jenis sumber daya (LUT, *flip-flop*, BRAM, atau DSP *slice*), N_{terpakai} sebagai jumlah sumber daya yang digunakan rancangan menurut laporan Vivado, dan N_{tersedia} sebagai jumlah sumber daya tersebut pada FPGA.

Selisih jumlah operasi dan utilisasi antara FXNLMS dan FXLMS digunakan untuk menjawab rumusan masalah ketiga.

3.5.5 Analisis Data Akhir

Analisis data akhir menggabungkan hasil Subbab 3.5.1, 3.5.2, dan 3.5.4. Kedua algoritma dibangun dengan filter adaptif, estimasi jalur sekunder, sinyal uji, dan parameter yang identik, sehingga satu-satunya perbedaan terdapat pada blok pembaruan koefisien. Oleh karena itu, selisih NR, kecepatan konvergensi, dan *steady-state error* antara FXNLMS dan FXLMS dinyatakan sebagai hasil dari mekanisme pembaruan koefisien tersebut. Selisih utilisasi sumber daya dinyatakan sebagai biaya komputasi dari mekanisme yang sama.

Hasil pada skenario daya tetap dan skenario daya berfluktuasi dibandingkan untuk melihat perubahan keunggulan FXNLMS ketika daya sinyal referensi berubah. Selisih kinerja tersebut kemudian disandingkan dengan selisih biaya komputasi. Perbandingan ini digunakan untuk menilai apakah keunggulan FXNLMS sepadan dengan biaya komputasi tambahannya.

[Halaman ini sengaja dikosongkan]

DAFTAR PUSTAKA

[1] T.-Y. Chen, J.-H. Yang, C.-L. Lai, and C.-T. Wei, "Low-frequency active noise control system based on feedback FXLMS," *Electronics*, vol. 14, no. 7, 2025, Art. no. 1442.

[2] Y. Han, *Active noise control system based on hybrid fixed filter method and FxLMS algorithm*, Nanjing University of Aeronautics and Astronautics, 2023.

[3] X. H. Xie, L. N. Zhou, and Y. J. Xie, *Design and simulation of active noise cancelling earphone system based on FXLMS algorithm*, Xi'an University of Posts and Telecommunications, 2022.

[4] A. Cesur, A. E. Canbey, H. Karsiyaka, and O. F. Maden, "Normalized FxLMS algorithm focused active noise cancellation (ANC) system approach in automotive industry," in *2025 16th International Conference on Electrical and Electronics Engineering (ELECO)*, Bursa, Türkiye, 2025, pp. 1–6. DOI: 10.1109/ELECO69582.2025.11329311

[5] Z. Ma and M. Lei, *Design of improved least mean square adaptive noise canceller*, Civil Aviation University of China.

[6] R. Zhang, "Investigation and evaluation of adaptive algorithms for multichannel active noise control system," M.Sc. dissertation, School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore, 2023. arXiv: 2308.15485. [Online]. Available: https://arxiv.org/abs/2308.15485

[7] D. Shi, B. Lam, W.-S. Gan, J. Cheer, and S. J. Elliott, *Active noise control in the new century: The role and prospect of signal processing*, 2023. arXiv: 2306.01425. [Online]. Available: https://arxiv.org/abs/2306.01425

[8] K. Sarkar, A. Lu, M. Mittal, Y. Zhuang, R. M. Corey, and A. C. Singer, *Latent FxLMS: Accelerating active noise control with neural adaptive filters*, 2025. arXiv: 2507.03854. [Online]. Available: https://arxiv.org/abs/2507.03854

[9] L. Zhang and X. Qiu, "Causality study on a feedforward active noise control headset with different noise coming directions in free field," *Applied Acoustics*, vol. 80, pp. 36–44, 2014. DOI: 10.1016/j.apacoust.2014.01.004

[10] Z. Li, Y. Yu, H. He, Y. Zhu, and R. C. de Lamare, *Study of switched step-size based filtered-x NLMS algorithm for active noise cancellation*, 2026. arXiv: 2601.16382. [Online]. Available: https://arxiv.org/abs/2601.16382

[11] A. Klemd, B. Klauer, J. Timmermann, and D. Sachau, "A flexible multi-channel feedback FxLMS architecture for FPGA platforms," in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*, Dublin, Ireland, 2021, pp. 319–326. DOI: 10.1109/FPL53798.2021.00063

[12] Y. Pang, S. Ou, Z. Cai, and Y. Gao, "Posterior error energy minimization based combined FxNLMS and FxLMS algorithm for active noise control," *IEEE Access*, vol. 12, pp. 30 754–30 764, 2024. DOI: 10.1109/ACCESS.2024.3368860

[13] M. Arezki, A. Namane, and A. Benallal, “Performance analysis of the fast-NLMS type algorithm,” in *Proceedings of the World Congress on Engineering 2017 (WCE 2017)*, vol. I, London, U.K., 2017, pp. 460–463.

[14] I. Kuon, R. Tessier, and J. Rose, “FPGA architecture: Survey and challenges,” *Foundations and Trends in Electronic Design Automation*, vol. 2, no. 2, pp. 135–253, 2008. DOI: 10.1561/1000000005

[15] AMD/Xilinx, *7 series FPGAs data sheet: Overview (DS180), v2.6.1*, September 8, 2020, 2020.

[16] A. G. Klemd, “Hardware acceleration of active noise control algorithms,” Dissertation (Dr.-Ing.) Helmut-Schmidt-Universität / Universität der Bundeswehr Hamburg, Hamburg, Germany, 2023. DOI: 10.24405/15013

[17] J. Thiemann, N. Ito, and E. Vincent, *DEMAND: A collection of multi-channel recordings of acoustic noise in diverse environments*, version 1.0, Zenodo, 2013. DOI: 10.5281/zenodo.1227121 [Online]. Available: https://zenodo.org/records/1227121

[18] D. Dean, S. Sridharan, R. Vogt, and M. Mason, “The QUT-NOISE-TIMIT corpus for the evaluation of voice activity detection algorithms,” in *Proceedings of INTERSPEECH 2010*, Makuhari, Japan, 2010, pp. 3110–3113. DOI: 10.21437/Interspeech.2010-774

[19] D. Snyder, G. Chen, and D. Povey, *MUSAN: A music, speech, and noise corpus*, 2015. arXiv: 1510.08484. [Online]. Available: https://www.openslr.org/17/